

# Problems with Forward Selection

Horacio Gomez-Acevedo

## Forward selection problems

Even though it is a very common practice to reduce the number of covariates using forward step-wise selection, it is in general not a good practice.

The following simulations are based on Dr. Harrell's notes <https://hbiostat.org/rmsc/multivar>

The setup is to create 8 independent random variables  $x_1, \dots, x_8$ , and a simple linear regression model for the independent variable  $l_p$  defined in terms of a given subset of variables

$$l_p = x_1 + x_2 + 0.5x_3 + 0.4x_7 + \varepsilon \text{ where } \varepsilon = N(0, \sigma^2)$$

The standard deviation is set to 2.5 by default but it can be change by the user. The model in R is written as

$$y \sim x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8$$

Note that we omit the coefficients in the formula. Then, we use Akaike's information criteria to carry on variable reduction.

```
require(MASS)    # provides stepAIC function
```

Loading required package: MASS

```
require(leaps)    # provides regsubsets function
```

Loading required package: leaps

```

sim <- function(n, sigma=2.5, method=c('stepaic', 'leaps'),
               pr=FALSE, prcor=FALSE, dataonly=FALSE) {
  method <- match.arg(method)
  if(uncorrelated) {
    x1 <- rnorm(n)
    x2 <- rnorm(n)
    x3 <- rnorm(n)
    x4 <- rnorm(n)
    x5 <- rnorm(n)
    x6 <- rnorm(n)
    x7 <- rnorm(n)
    x8 <- rnorm(n)
  }
  else {
    x1 <- rnorm(n)
    x2 <- x1 + 2.0 * rnorm(n)           # was + 0.5 * rnorm(n)
    x3 <- rnorm(n)
    x4 <- x3 + 1.5 * rnorm(n)
    x5 <- x1 + rnorm(n)/1.3
    x6 <- x2 + 2.25 * rnorm(n)         # was rnorm(n)/1.3
    x7 <- x3 + x4 + 2.5 * rnorm(n)     # was + rnorm(n)
    x8 <- x7 + 4.0 * rnorm(n)         # was + 0.5 * rnorm(n)
  }
  z <- cbind(x1,x2,x3,x4,x5,x6,x7,x8)
  if(prcor) return(round(cor(z), 2))
  lp <- x1 + x2 + .5*x3 + .4*x7
  y <- lp + sigma*rnorm(n)
  if(dataonly) return(list(x=z, y=y))
  if(method == 'leaps') {
    s <- summary(regsubsets(z, y))
    best <- which.max(s$adjr2)
    xvars <- s$which[best, -1] # remove intercept
    ssr <- s$rss[best]
    p <- sum(xvars)
    xs <- if(p == 0) 'none' else paste((1 : 8)[xvars], collapse='')
    if(pr) print(xs)
    ssesw <- (n - 1) * var(y) - ssr
    s2s <- ssesw / (n - p - 1)
    yhat <- if(p == 0) mean(y) else fitted(lm(y ~ z[, xvars]))
  }
  f <- lm(y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8)
  if(method == 'stepaic') {

```

```

g <- stepAIC(f, trace=0)
p <- g$rank - 1
xs <- if(p == 0) 'none' else
  gsub('[ <br>+x]', '', as.character(formula(g))[3])
if(pr) print(formula(g), showEnv=FALSE)
ssesw <- sum(resid(g)^2)
s2s <- ssesw/g$df.residual
yhat <- fitted(g)
}

# Set SSEsw / (n - gdf - 1) = true sigma^2
gdf <- n - 1 - ssesw / (sigma^2)
# Compute root mean squared error against true linear predictor
rmse.full <- sqrt(mean((fitted(f) - lp) ^ 2))
rmse.step <- sqrt(mean((yhat - lp) ^ 2))
list(stats=c(n=n, vratio=s2s/(sigma^2),
             gdf=gdf, apparentdf=p, rmse.full=rmse.full, rmse.step=rmse.step),
      xselected=xs)
}

rsim <- function(B, n, method=c('stepaic', 'leaps')) {
  method <- match.arg(method)
  xs <- character(B)
  r <- matrix(NA, nrow=B, ncol=6)
  for(i in 1:B) {
    w <- sim(n, method=method)
    r[i,] <- w$stats
    xs[i] <- w$xselected
  }
  colnames(r) <- names(w$stats)
  s <- apply(r, 2, median)
  p <- r[, 'apparentdf']
  s['apparentdf'] <- mean(p)
  print(round(s, 2))
  print(table(p))
  cat('Prob[correct model]=', round(sum(xs == '1237')/B, 2), '\n')
}

```

You can add options to executable code like this

```
[1] 4
```

The `echo: false` option disables the printing of code (only output is displayed).